Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree

Splay Tree:

- A **Splay Tree** is a **self-adjusting Binary Search Tree (BST)**
- **Whenever a node is accessed, move that node to the root**
- This operation is called **splaying**

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: salient features

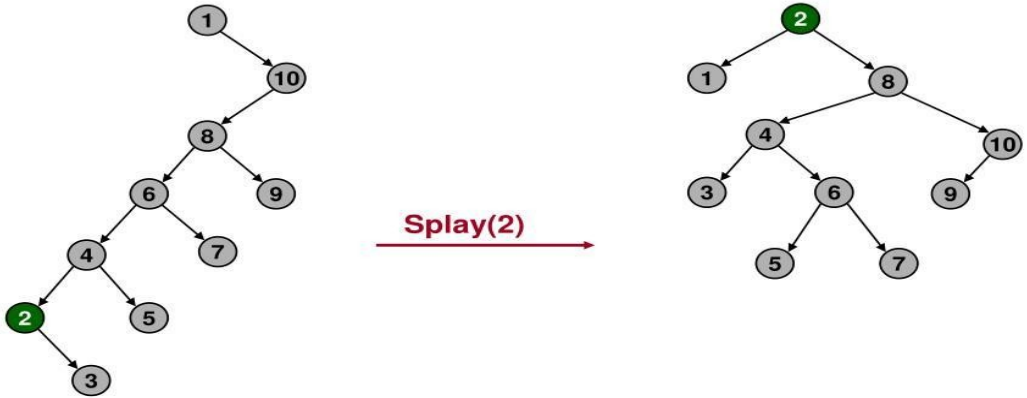
- **No Extra Metadata:** They require **no balance accounting data** (like color bits in red-black trees or height numbers in AVL trees), saving memory.
- **Locality of Reference:** If we access a piece of data once, it moves to the top, making **subsequent lookups of the same data instantaneous**.
- **Efficiency:** While a single worst-case operation can take time proportional to the number of elements, a sequence of m operations is guaranteed to run in time proportional to m * log(n, 2).
- **Dynamic Adaptation:** The tree structure continuously morphs to fit the **actual access pattern** of the workload rather than enforcing a rigid shape.

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree

Splay Example

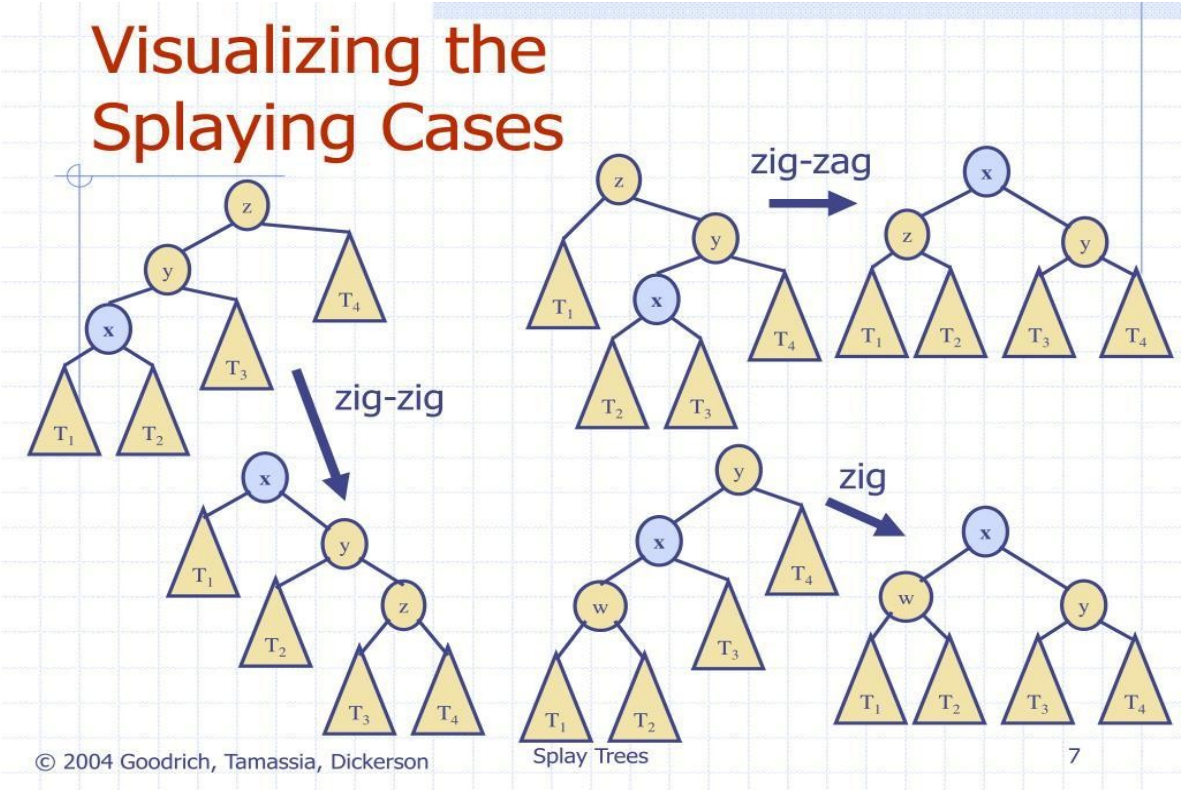
Apply Splay(2, S) to tree S:



28

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree



© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: operations

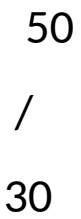
- **Zig**
- **Zig-Zig**
- **Zig-Zag**

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG

- Node accessed is the child of the root

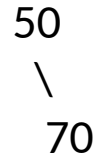
Suppose we access 30:



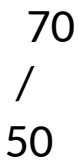
Perform a **right rotation**:



Suppose we access 70



perform a **left rotation:**



Zig rule

Node is child of root

↓

Perform one rotation

↓

Node becomes root

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZIG (LL case)

- node and its parent are both on the **same side**

```
      50
     /
    30
   /
  20
```

We want to access 20.

Here:

20 is the **left child** of 30

30 is the **left child** of 50

So this is **Left-Left (LL)**.

First rotation

Rotate 50 right:

```
      30
     /  \
    20   50
```

Second rotation

Rotate 30 right:

```
    20
      \
       30
         \
          50
```

Zig-ZIG rule

LL => two right rotations

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZIG (RR case)

- node and its parent are both on the **same side**

```
20
  \
   30
    \
     50
```

Access 50.

50 is the **right child** of 30

30 is the **right child** of 20

So this is RR case

First rotation

Rotate 30 left:

```
    30
   /  \
  20   50
```

Second rotation

Rotate 50 left:

```
    50
   /
  30
 /
20
```

Zig-ZIG rule

LL => two right rotations

RR => two left rotations

© Confidential Information of PES University. All Rights Reserved - 2026

Splay Tree: ZIG-ZAG

- node and its parent are both on the **opposite side**

Consider:

```
    50
   /
  20
   \
    30
```

30 is the **right child** of 20

20 is the **left child** of 50

Therefore:

Left-Right (LR) configuration.

First rotation

Rotate 20 left:

```
    50
   /
  30
 /
20
```

Second rotation

Rotate 50 right:

```
    30
   / \
  20  50
```

So 30 becomes the root.

Zig-Zag rule

LR => one left; one right

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)